

A PROOFS

A.1 Proof of Theorem 1

Given an input parameter pair (ϵ, μ) , by the definition of (ϵ, ρ) -neighbors, for any vertex v , the number of its (ϵ, ρ) -neighbors lies in the interval $[d_{\epsilon+\rho}[v], d_{\epsilon-\rho}[v]]$. By the definition of core vertices, the core vertex set produced by the ρ -approximate SCAN with parameters (ϵ, ρ, μ) satisfies

$$V_{\epsilon+\rho, \mu}^{core} \subseteq V_{\epsilon, \rho, \mu}^{core} \subseteq V_{\epsilon-\rho, \mu}^{core}$$

where $V_{\epsilon, \rho, \mu}^{core}$ denotes the core vertex set under parameters (ϵ, μ) . According to Properties 2 and 3 of SCAN, the inclusion relationships are preserved across the corresponding clustering results. Therefore, the theorem holds.

A.2 Proof of Theorem 2.

After uniform discretization of the similarity range, consider an input parameter pair (ϵ, μ) . If ϵ lies exactly on a discretization boundary, the exact clustering result can be obtained by directly applying an exact SCAN algorithm. Otherwise, suppose that $\epsilon \in [(i-1)/\delta, i/\delta)$. In this case, we return the clustering result $C_{(i-1)/\delta, \mu}$, i.e., the clustering result under parameter pair $(\frac{i-1}{\delta}, \mu)$. By construction, we have

$$\epsilon - \frac{1}{\delta} < \frac{i-1}{\delta} < \epsilon + \frac{1}{\delta},$$

which implies

$$(\epsilon - \frac{1}{\delta}, \mu) \prec (\frac{i-1}{\delta}, \mu) \prec (\epsilon + \frac{1}{\delta}, \mu).$$

Therefore, by the definition of ρ -approximate SCAN and the nested property of SCAN, the returned clustering is a ρ -approximate result with $\rho = \frac{1}{\delta}$. The theorem thus holds.

A.3 Proof of Theorem 3

In Algorithm 2, each vertex is processed at most once in each tree i , and each tree node is processed exactly once during tree construction. Let $\alpha(\cdot)$ denote the inverse Ackermann function. For each edge in $E_{\frac{i-1}{\delta}, 2}^{\neq}$, computing $R_{children}^v$ and completing the NC list each take $O(1)$ time. For each edge in $E_{\frac{i-1}{\delta}, 2}$, the union operation for constructing clusters takes $O(\alpha(n))$ amortized time. In addition, processing all tree nodes across the δ trees requires $O(\delta \cdot n_{tree}^{avg})$ time.

Overall, let m_{tree}^{avg} denote the average value of $m_{\frac{i-1}{\delta}, 2}$, the time complexity of Algorithm 2 is bounded by

$$O\left(\delta \cdot (n_{tree}^{avg} + m_{tree}^{\neq, avg} + \alpha(n) \cdot m_{tree}^{avg})\right).$$

A.4 Proof of Theorem 4

Suppose that v is core under (ϵ, μ) . Let ϵ' be the largest similarity threshold for which $d_{\epsilon'}[v] \geq \mu$, and let $\mu' = d_{\epsilon'}[v]$. Such a maximum exists because $N[v]$ is finite and neighbors are included when their similarity equals the threshold. Thus, $\epsilon' \geq \epsilon$ and $\mu' \geq \mu$. No feasible pair can strictly dominate (ϵ', μ') : a larger similarity threshold contradicts the maximality of ϵ' , while a larger cardinality threshold at ϵ' exceeds $d_{\epsilon'}[v]$. Hence, $(\epsilon', \mu') \in PPT(v)$ and $(\epsilon, \mu) \preceq (\epsilon', \mu')$.

Then, suppose that $(\epsilon, \mu) \preceq (\epsilon', \mu')$ for some $(\epsilon', \mu') \in PPT(v)$. Since $\epsilon' \geq \epsilon$, we have $N_{\epsilon'}[v] \subseteq N_{\epsilon}[v]$. Therefore, $d_{\epsilon}[v] \geq d_{\epsilon'}[v] \geq \mu' \geq \mu$, so v is core under (ϵ, μ) . The theorem follows.

A.5 Proof of Theorem 5

Consider a vertex v with degree $d(v) \geq 1$ and δ discretized similarity intervals. If $N(v) = \{u\}$, then v has at least one PPT $(\sigma(u, v), 2)$, and thus $|PPT(v)| \geq 1$. If $d(v) \leq n_{interval}^v$, the maximum number of PPTs of v is $d(v)$, which occurs when all similarities $\sigma(v, u_1)$ and $\sigma(v, u_2)$ are belong to distinct interval for any $u_1, u_2 \in N(v)$ with $u_1 \neq u_2$. If $d(v) > n_{interval}^v$, the maximum number of PPTs of v is $n_{interval}^v$, which occurs when for each interval $i \in [1, n_{interval}^v]$, there exists a neighbor $u \in N(v)$ such that $\sigma(u, v) \in [(i-1)/\delta, i/\delta)$. Therefore, for any vertex v with $d(v) \geq 1$, we have

$$1 \leq |PPT(v)| \leq \min\{d(v), n_{interval}^v\}.$$

The theorem holds.

A.6 Proof of Theorem 6

First, we prove correctness, i.e., the connectivity among all cluster slices can be correctly recovered at query time. Assume that two cluster slices $S_1(\epsilon_1, \mu_1)$ and $S_2(\epsilon_2, \mu_2)$ are connected under an input parameter pair (ϵ, μ) , where $\epsilon \leq \min\{\epsilon_1, \epsilon_2\}$ and $\mu \leq \min\{\mu_1, \mu_2\}$.

We consider the following three cases.

Case (1): S_1 and S_2 can be connected through other cluster slices. In this case, the connectivity between S_1 and S_2 can be recovered by transitively connecting them via these intermediate cluster slices using the stored connectivity.

Case (2): S_1 and S_2 cannot be connected through other cluster slices, and there exists at least one edge (u, v) between them such that $\sigma(u, v) \geq \min\{\epsilon_1, \epsilon_2\}$. In this case, (u, v) satisfies the conditions of Strategies 2–5 and is therefore stored in PPT-Index, which allows S_1 and S_2 to be directly connected.

Case (3): S_1 and S_2 cannot be connected through other cluster slices, and for all edges (u, v) between them, $\sigma(u, v) < \min\{\epsilon_1, \epsilon_2\}$. By Strategy 3, none of these edges is stored. However, since S_1 and S_2 are connected under (ϵ, μ) , there must exist an edge (u, v) such that $\sigma(u, v) \geq \epsilon$. Consequently, u and v belong to cluster slices $S_3(\sigma(u, v), \mu_3)$ and $S_4(\sigma(u, v), \mu_4)$, respectively, and the edge (u, v) is stored to connect S_3 and S_4 . Thus, S_1 and S_2 can be connected transitively through S_3 and S_4 .

In all cases, the connectivity between S_1 and S_2 can be correctly recovered. Therefore, the correctness holds.

Then, we prove that the storage of connections is non-redundant.

The edges between cluster slices with shared vertices are pruned by Strategy 2. If the number of the rest of edges is not minimal, there will be redundant edges. Assume, for the sake of contradiction, that there exist two redundant edges e_1 and e_2 contributing the same connectivity. We denote the pair of cluster slices connected by e_1 as S_1 and the pair connected by e_2 as S_2 . If $S_1 = S_2$, according to Strategy 4, upon storing e_1 , e_2 will not be stored. On the other hand, if $S_1 \neq S_2$, for Strategy 5, only one of e_1 and e_2 will be stored. These two cases contradict the assumption. Thus, the storage of connections is non-redundant.

Finally, we prove the claimed bound on the number of stored connections. Fix a query (ϵ, μ) and let $Q = V_{\epsilon, \mu}^{core}$. Assign each stored

Table 4: Dataset statistics for complexity analysis ($\delta = 10$, $K = 10^3$, $M = 10^6$).

Abbr.	Graph	n	m	d_{avg}	n_{tree}^{avg}	m_{tree}^{avg}	$m_{tree}^{\neq, avg}$	$n_{interval}^{avg}(V)$	$ S $	$\sum_R R.Con $	$ PPT $
AM	amazon	334.9K	925.9K	5.5	163.15K	275.02K	222.88K	4.38	397.68K	289.22K	145
YT	youtube	1.1M	3.0M	5.3	249.11K	202.35K	139.87K	1.98	476.69K	241.69K	174
SK	skitter	1.7M	11.1M	13.1	386.05K	453.24K	348.19K	2.05	786.50K	431.72K	681
PK	pokec	1.6M	22.3M	27.3	227.45K	456.91K	399.89K	1.25	439.94K	224.98K	380
TC	topcats	1.8M	28.5M	31.8	331.43K	724.71K	589.77K	1.67	781.17K	471.90K	748
LJ	livejournal	4.0M	34.7M	17.3	1.17M	3.92M	3.42M	2.63	2.35M	1.49M	4100
OK	orkut	3.1M	117.2M	76.3	525.01K	3.55M	3.41M	1.54	889.40K	554.65K	1157
IC	indochina	7.4M	151.0M	40.7	3.49M	46.81M	10.50M	4.24	3.09M	2.04M	4436
UK2	uk2002	18.5M	261.8M	28.3	8.64M	44.21M	24.97M	4.21	9.53M	6.49M	4255
UK5	uk2005	39.5M	783.0M	39.7	20.87M	202.51M	99.84M	4.76	20.85M	14.68M	5274
IT	it2004	41.3M	1.0B	49.8	20.38M	153.91M	79.73M	4.44	19.01M	12.76M	12649
FS	friendster	65.6M	1.8B	55.1	6.87M	10.16M	8.36M	0.94	14.59M	7.08M	840

connection satisfying $u, v \in Q$ to the similarity interval in which it was last inserted or replaced.

Based on strategies 2-5 and Algorithm 4, for each interval i , the connectivity union-find structure is maintained as the construction threshold μ' decreases, without splitting or reinitializing its components. Every insertion or replacement occurs only when the endpoints belong to distinct components, which are then united. Hence, the edges corresponding to these successful unions form a forest. The qualifying connections assigned to interval i form a subset of these edges and therefore also form a forest.

Let Q_i be the vertices in Q that participate in the construction for interval i . The number of qualifying connections assigned to this interval is at most $|Q_i|$. Since each vertex $v \in Q$ participates in at most $n_{interval}^v$ intervals, the total number of qualifying connections is bounded by

$$\sum_i |Q_i| \leq \sum_{v \in Q} n_{interval}^v = n_{interval}^{avg}(Q) \cdot |Q|.$$

Substituting $Q = V_{\epsilon, \mu}^{core}$ proves the claimed bound.

A.7 Proof of Theorem 7

Algorithm 4 iterates over parameter pairs $p = (i, \mu)$, with i ranging from δ down to 1. During cluster slice computation, each vertex v is added to the temporary set V' at most $n_{interval}^v$ times. The total cost of processing vertices is bounded by $O(n_{interval}^{avg}(V) \cdot n)$.

For each interval i , every edge $(u, v) \in E_{i-1, 2}$ is involved in at most one union operation when computing the connected components. Computing the Con lists further requires uniting vertices within existing cluster slices and processing edges across different cluster slices. Overall, the number of union operations for each interval is bounded by $O(m_{i-1, 2})$.

In addition, computing and collecting $PPT(v)$ and $\mathcal{P}(v)$ for all vertices requires $O(n_{interval}^{avg}(V) \cdot n)$ time. Initializing the entries for all PPTs requires $O(|PPT|)$ time. Since $|PPT| \leq n_{interval}^{avg}(V) \cdot n$, this cost is subsumed by the vertex-processing cost.

Overall, the time complexity of Algorithm 4 is bounded by

$$O(n_{interval}^{avg}(V) \cdot n + \delta \cdot \alpha(n) \cdot m_{tree}^{avg}).$$

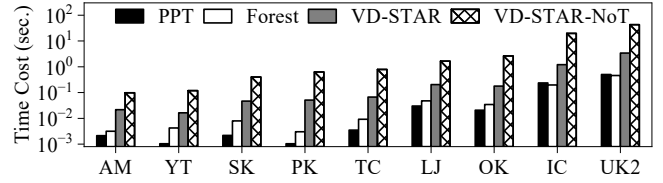


Figure 15: Comparison with VDSTAR on clustering efficiency.

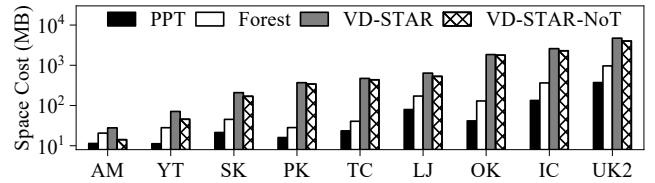


Figure 16: Comparison with VDSTAR on space cost.

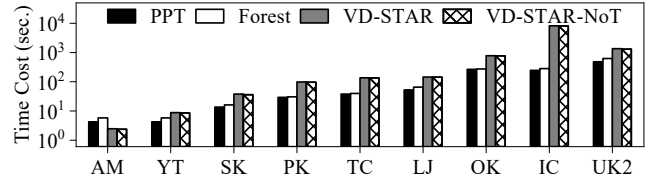


Figure 17: Comparison with VDSTAR on build cost.

B COMPARISON WITH VD-STAR[47] AND ITS VARIANTS

We obtained the source code of VD-STAR (including VD-STAR and VD-STAR-NoT) kindly provided by its authors and conducted additional comparisons. Under our experimental environment, VD-STAR ran out of memory during index construction on several large datasets (UK5, IT, and FS). We report the results of VD-STAR on the datasets where its index can be successfully constructed.

Figures 15, 16, and 17 compare clustering efficiency, space cost, and construction time, respectively. Both proposed indexes outperform VD-STAR and VD-STAR-NoT across all tested datasets in these metrics. VD-STAR prioritizes update efficiency, while its clustering acceleration follows the common indexing framework, similar to BOTBIN. In contrast, our indexes exploit nested cluster relationships to reduce storage redundancy and query processing costs.

C DATASET STATISTICS FOR COMPLEXITY ANALYSIS

We report the dataset-specific values of the quantities used in our complexity analysis. The details is shown in Table 4.